

Full-Stack Home Assignment: FieldNet Monitor

Sep 28, 2026 ·

Build the server, state engine and operator console for a sensor network whose devices are unreliable. You compute every gateway's and sensor's situation from the evidence the devices send, move it between situations by the rules below, and expose it through an API and a UI. Budget: about 10 hours. AI coding tools are allowed.

The scenario

FieldNet runs **gateways** at remote sites. Each gateway is assigned some **sensors**; a sensor may be covered by more than one gateway. Every cycle, a gateway checks each of its sensors, reports what it found, and uploads any readings in batches. The server processes batches in the background.

Operators use a console to answer three questions: which gateways are really working, which sensors are really alive, and why. The devices make that hard. They time out, retry, send the same batch twice, send a batch before the report that mentions it, lose their credentials, keep heart beating while collecting nothing, and ignore commands. Your system must tell the truth anyway, including "we don't know".

Situations

The axes are stored and shown separately, but they are not independent. The dependencies run one way: gateway status and command state decide what each gateway contributes (its coverage class). Coverage decides which evidence counts for a sensor. Evidence decides the sensor's collection and lifecycle. A sensor's state never changes a gateway.

The values in code are the exact strings the API returns.

Axis	Values
Gateway status	new, spare, connected, stale, disconnected, suspended, retired
Gateway command state	running, stop_pending, stopped, stop_failed, resume_pending, resume_failed
Gateway coverage class (derived)	available, stopped, recoverable, dead
Sensor coverage (derived)	available, stopped, recoverable, none
Sensor lifecycle	pending, active, dormant, sampling, retired, decommissioned
Sensor lifecycle reason	no_readings or no_live_coverage when retired; the operator's text when decommissioned; otherwise null

Axis	Values
Sensor collection	readings, no_readings, could_not_read, timed_out, not_checked, collection_stopped
Batch processing	received, processed, partially_processed, retrying, quarantined

Every change of any axis appends a timeline entry. Entries are never edited or deleted.

Time and evidence

Three clocks. Every piece of evidence has an *event time* set by the device: a heartbeat's `sent_at`, a cycle's `finished_at`, a reading's `taken_at`, an acknowledgement's `acked_at`. It also has a *received time* (the server clock when the request arrived), and a batch has a *processed time*. A device timestamp more than 5 minutes after its received time is invalid. Which clock each rule uses:

Rule	Clock
<code>last_heartbeat_at</code>	Received time of the latest heartbeat
<code>last_qualifying_at</code> , the 12-hour Stale rule, <code>disconnected_since</code> and recovery from Disconnected	Event time
Sensor days, quiet counts, wake-ups and sampling checks	Event time
Operator actions, coverage changes, configuration changes, command timeouts	Received time (server clock)
When a transition is <i>recorded</i>	Server clock. Each timeline entry carries both <code>effective_at</code> and <code>recorded_at</code>

Qualifying evidence. This is evidence that a gateway is collecting, not just running:

- a cycle with `session: "ok"` and at least one `readings` or `no_readings` outcome, at its `finished_at`;
- a batch that ends `processed` or `partially_processed` with at least one accepted reading, at its latest accepted `taken_at`.

Heartbeats, batches still `received` or `retrying`, and cycles with `session: "auth_failed"` never qualify.

Sensor days. Days are UTC dates. A day is *complete* at 00:00 UTC on the next day, and only complete days are counted. Each sensor-day is classified as follows, first match wins:

- **Reading day:** at least one accepted reading was taken that day, from any gateway.
- **Unresolved:** a `readings` outcome that day points to a batch that is not yet resolved. The batch resolves when it ends `processed`, `partially_processed` or `quarantined`, or when 24 hours

pass after the cycle's `finished_at` without it arriving. A quarantined or missing batch turns that outcome into `could_not_read`.

- **Quiet day:** at least one `no_readings` outcome that day from a gateway whose coverage class was `available` at that time.
- **Unchecked day:** everything else. That includes `could_not_read` and `timed_out` outcomes, and silence.

Any evaluation that depends on an unresolved day waits until the day resolves.

Available time. Time during which the sensor's coverage is `available`. Dormant waits and sampling windows are measured in available time, not calendar time. When coverage stops being `available`, they pause. `next_evaluation_at` is the server time when the current wait or window ends if coverage stays `available`; it is `null` while paused.

Latest collection. Take each gateway whose coverage class is `available`. Use its most recent cycle outcome for the sensor, but only if that cycle finished within the last 24 hours. Among those outcomes, pick by this precedence: `readings`, `no_readings`, `could_not_read`, `timed_out`. If no outcome qualifies, the collection is `collection_stopped` when the sensor's coverage is stopped, otherwise `not_checked`. A reading accepted from a batch that no cycle mentions counts as a `readings` outcome from its gateway, timed at the reading's `taken_at`.

Coverage

Each gateway has a coverage class, derived from its status and command state:

Gateway	Coverage class
connected with command state <code>running</code> , <code>stop_pending</code> or <code>stop_failed</code> (no proof it stopped)	<code>available</code>
connected with command state <code>stopped</code> , <code>resume_pending</code> or <code>resume_failed</code> (no proof it resumed)	<code>stopped</code>
<code>new</code> , <code>stale</code> or <code>disconnected</code> , whatever the command state	<code>recoverable</code>
<code>suspended</code> or <code>retired</code>	<code>dead</code>

A sensor's coverage is the best class among the gateways covering it, in this order: `available`, `stopped`, `recoverable`. It is `none` if every covering gateway is `dead`, or if none is assigned.

Sensor coverage	Collection	Lifecycle	Timers
<code>available</code>	Latest collection rule	Evaluated by the sensor rules	Run

Sensor coverage	Collection	Lifecycle	Timers
stopped	collection_stopped	Unchanged, but readings still wake it	Paused
recoverable	not_checked	Unchanged, but readings still wake it	Paused
none	not_checked	pending and decommissioned stay; any other state becomes retired with reason no_live_coverage at once	Paused

Recompute coverage whenever anything that feeds it changes: a gateway's status or command state, or a sensor's assignment. It is not only recomputed on assignment. A retired (no_live_coverage) sensor becomes pending as soon as its coverage is anything other than none. That includes a covering gateway leaving suspended, or a new gateway being assigned. Its counters reset.

Process every gateway request in this order: first update the gateway from the request, then recompute coverage, then apply the request's evidence to sensors. That way, a Stale gateway's first good cycle counts as a check.

Gateway transitions

From	To	Trigger
new	connected	Qualifying evidence
new	spare	Operator mark_spare. Allowed only when the gateway covers no sensors; otherwise 409
spare	new	A sensor is assigned to it
connected	stale	12 hours have passed since last_qualifying_at
stale	connected	Qualifying evidence whose event time is within the last 12 hours. Older, late evidence updates nothing
new, connected, stale	disconnected	A heartbeat or cycle with session: "auth_failed". disconnected_since is that request's event time. Later

From	To	Trigger
		failures in the same disconnection keep the first time
disconnected	connected	Qualifying evidence whose event time is after <code>disconnected_since</code> . A heartbeat with <code>session: "ok"</code> is not enough, and neither is an old batch re-sent after the failure
Any except <code>retired</code>	suspended	Operator suspend with a reason. While suspended, automatic rules never change the status, but <code>last_heartbeat_at</code> , <code>last_qualifying_at</code> and auth failures keep updating
suspended	Derived	Operator unsuspend. The status is derived from the evidence: <code>disconnected</code> if the latest auth failure is newer than <code>last_qualifying_at</code> ; otherwise <code>connected</code> if <code>last_qualifying_at</code> is within 12 hours; otherwise <code>stale</code> ; new if there has never been qualifying evidence
Any	retired	Operator retire. Terminal. Later gateway requests get 403 and change nothing

A new gateway with no evidence stays new indefinitely. It never becomes spare on its own.

Stop and resume

Commands are ordered: each gets a per-gateway sequence number, and the latest command issued is the desired state. `GET /gw/v1/commands` returns only the latest command, and only while it is unacknowledged. An acknowledgement of an older, superseded command is accepted (204) and recorded in the timeline, but never changes the command state.

From	To	Trigger
running	stop_pending	Operator stop
stop_pending	stopped	The gateway acknowledges that stop
stop_pending	stop_failed	No acknowledgement within 10 minutes of the stop being issued (server clock)
stop_failed	stopped	A late acknowledgement of that stop, if it is still the latest command
stop_pending, stopped, stop_failed	resume_pending	Operator resume. This supersedes the stop
resume_pending	running	The gateway acknowledges that resume
resume_pending	resume_failed	No acknowledgement within 10 minutes
resume_failed	running	A late acknowledgement of that resume
resume_pending, resume_failed	stop_pending	Operator stop. This supersedes the resume

stop while the state is stop_pending or stopped, and resume while it is running, return 409.

The collecting-after-stop flag. A *stop period* runs from the `acked_at` of a stop to the `acked_at` of the resume that superseded it. If there is no such resume yet, it is open-ended. Any accepted reading from that gateway taken inside a stop period is kept and counted, raises the `collecting_after_stop` flag on the gateway, and adds a timeline entry. The flag is shown while the current stop period has such readings, and clears when a resume is acknowledged. Readings taken before the stop's `acked_at` or after the resume's `acked_at` never raise it.

Sensor transitions

Quiet days, waits and windows need the sensor's coverage to be available. A reading counts whatever the coverage: readings are facts, even from a stopped gateway. The Coverage section covers everything else. Thresholds are configuration, with these defaults:

Setting	Default
Quiet days before Dormant	14
Dormant wait	14 days of available time

Setting	Default
Sampling window	72 hours of available time
Sampling cycles before Retired	2
Stale after	12 hours
Command timeout	10 minutes
Batch deadline	24 hours

From	To	Trigger	Effective at
pending	active	The first reading day	The reading's taken_at
active	dormant	The 14th quiet day since the last reading day completes. The reading day itself does not count. A reading day resets the count to 0; unchecked days neither count nor reset it	00:00 UTC after that 14th quiet day
dormant	sampling	14 days of available time have passed in dormant	When they complete
sampling	active	A reading taken inside the window	The reading's taken_at
sampling	dormant	72 hours of available time end. The window had at least one no_readings outcome with finished_at inside it, from a gateway whose coverage class was available at that time,, and no reading. sampling_cycles_done goes up by 1	Window end

From	To	Trigger	Effective at
sampling	retired (no_readings)	As above, and this was cycle number 2	Window end
sampling	sampling (new window)	The window ends with no no_readings outcome inside it. It does not count as a cycle	Window end
dormant, retired (no_readings)	active	A reading taken while in that state	The reading's taken_at
Any except decommissioned	decommissioned	Operator decommission with a reason. Terminal; any later action returns 409	Server clock

Counters: quiet_checked_days resets to 0 on every reading day and on entering pending.
sampling_cycles_done resets to 0 on entering active or pending.

When several rules fire at the same instant, apply them in this order: decommission, loss of coverage, readings, then timers.

Late data and corrections

A sensor's lifecycle is a deterministic function of four inputs: its evidence by event time, its coverage history, operator actions, and the configuration in force at each moment. If evidence arrives with an event time earlier than a transition already recorded, recompute the sensor from that event time to now.

- If the recomputed history matches, record nothing.
- If it differs, append one correction entry. It records the previous current state, the corrected current state, the recomputed transitions in between, and the evidence id that caused it. Earlier entries are never edited. The state API returns the corrected current state. After a correction, lifecycle_since and status_since are the effective_at of the latest transition in the recomputed history, not the time of the correction.
- A late reading does not simply make the sensor Active now. Example: a sensor went dormant on day 15, and on day 40 a batch arrives with a reading taken on day 2. The recompute moves every later timer, but the sensor may still correctly be dormant or sampling today.
- A reading id is globally unique. The first processed version wins. A later reading with the same id and the same content is ignored. A later reading with the same id and different content is quarantined with reason conflicting_duplicate.

Gateway status is not recomputed backwards. Late evidence updates last_qualifying_at only when it is newer than the current value.

Worked example

Sensor S is covered only by gateway G. G is connected and running, and reports S once a day at 12:00 UTC. Day n means 00:00 UTC on the n -th day after day 0.

When	Evidence	S after
Day 0, 12:00	readings; the batch is processed at 12:01	active from day 0, 12:00
Days 1–14, 12:00	no_readings each day	active, quiet_checked_days 1–14 as each day completes
Day 15, 00:00	Day 14 completes	dormant; next_evaluation_at is day 29, 00:00
Day 29, 00:00	14 days of available time have passed	sampling; the window ends day 32, 00:00
Days 29–31	no_readings each day	sampling
Day 32, 00:00	The window ends with checks and no reading	dormant, sampling_cycles_done 1
Day 46, 00:00	The wait ends	sampling
Day 49, 00:00	The second window ends with checks and no reading	retired (no_readings)

Three variations on the same run:

- **Disconnection mid-window.** G reports `auth_failed` on day 30 at 12:00 and sends a good cycle on day 37 at 12:00. The first window has used 36 of its 72 hours, so it pauses at 36 hours remaining. G's good cycle on day 37 at 12:00 makes it `connected` again, the window resumes, and it ends on day 39 at 00:00. S stays `sampling` throughout, and its collection shows `not_checked` while paused.
- **A late reading.** On day 20, G uploads a batch holding a reading taken on day 9. After recompute, days 10–19 are 10 quiet days, so S should be `active`. A `correction` entry records `dormant` → `active`, and the day-15 `dormant` entry stays.
- **An unresolved day.** On day 14, G reports readings but the batch never arrives. Day 14 is unresolved until day 15 at 12:00, then counts as `unchecked`. S goes `dormant` only after another quiet day completes: day 16 at 00:00.

The fixed API

Implement these endpoints exactly as written. We test against them.

1. All bodies are JSON and all times are ISO 8601 UTC.
2. Errors are `{"error": "<code>", "detail": "<text>"}`.

- Invalid bodies return 422. Conflicts and illegal actions return 409. An unknown gateway, sensor, batch or command id in a path or body returns 404.

Gateway API

Devices call these with Authorization: Bearer <token>. The token identifies the gateway. An unknown token gets 401; a retired gateway gets 403.

Endpoint	Body	Responses
POST /gw/v1/heartbeat	{sent_at, session}, where session is ok or auth_failed	204
POST /gw/v1/cycles	{cycle_id, started_at, finished_at, session, results: [{sensor_id, outcome, batch_id}]}. outcome is one of readings, no_readings, could_not_read, timeout; timeout shows as timed_out in collection. batch_id is required when outcome is readings and must be absent otherwise. With session: "auth_failed", every outcome is recorded as could_not_read	202 with {"ignored": [sensor_id]}, listing results for sensors this gateway does not cover, which are ignored. A repeat with an identical body returns 200 and changes nothing; the same cycle_id with a different body returns 409 cycle_conflict
PUT /gw/v1/batches/{batch_id}	{sensor_id, readings: [{reading_id, taken_at, value, unit}]}	202 within 1 second; processing happens in the background. An identical repeat returns 200; a different body returns 409 batch_conflict. A batch may arrive before or after the cycle that mentions it, or with no cycle at all. batch_id is globally unique (gateways generate UUIDs), so the same id from a different gateway is also 409 batch_conflict
GET /gw/v1/commands	none	200 {"commands": [{command_id, seq, type, issued_at}]}, where type is stop or resume. At most one command: the latest, while unacknowledged

Endpoint	Body	Responses
POST /gw/v1/commands/{command_id}/ack	{acked_at}	204, idempotent. 404 if the command is not this gateway's

The server may answer 503 with Retry-After when overloaded. Gateways retry any non-2xx, so every write must be safe to repeat.

Batch processing

- A reading is **invalid** if a field is missing, value is not a number, unit is wrong for the sensor type, or taken_at is more than 5 minutes after the batch's received time.
- Units by sensor type: temperature uses C, rain uses mm, wind uses m/s.
- If some readings are invalid, the batch is **partially_processed**: the valid readings are accepted, and the invalid ones are quarantined with a reason.
- If every reading is invalid, or the sensor is unknown, the batch is **quarantined**.
- If processing itself fails, the batch is **retrying**. Attempt k (from 1) waits $2^{(k-1)}$ minutes on the server clock before the next one. After the 5th failed attempt, the batch is quarantined with reason **processing_failed**. A retried batch never double-counts.

Operator and state API

The console and our tests call these.

Endpoint	Purpose
POST /api/v1/gateways {gateway_id, name}	Register a gateway. Returns 201 {token}. A duplicate id returns 409
POST /api/v1/sensors {sensor_id, type}	Register a sensor as pending with no coverage
PUT /api/v1/sensors/{id}/coverage {gateway_ids}	Replace the set of gateways covering the sensor. A retired gateway id returns 409
POST /api/v1/gateways/{id}/actions {action, reason}	action is suspend (reason required), unsuspend, mark_spare, retire, stop or resume. Returns the new gateway state
POST /api/v1/sensors/{id}/actions {action, reason}	decommission, reason required
GET /api/v1/gateways/{id}	{gateway_id, name, status, status_since, command_state, coverage_class, last_heartbeat_at, last_qualifying_at, disconnected_since, flags}. flags is a list of strings; the only flag is collecting_after_stop
GET /api/v1/sensors/{id}	{sensor_id, type, lifecycle, lifecycle_reason, lifecycle_since, collection, coverage,

Endpoint	Purpose
	quiet_checked_days, sampling_cycles_done, next_evaluation_at}
GET /api/v1/gateways/{id}/timeline, GET /api/v1/sensors/{id}/timeline	Entries, oldest first: {seq, kind, axis, from, to, effective_at, recorded_at, rule, evidence_ids}. kind is transition, correction, action, ack, flag or rule_change
GET /api/v1/batches/{id}	{processing, attempts, accepted_count, quarantined: [{reading_id, reason}]}

Test endpoints

These exist only when TEST_MODE=1; otherwise they return 404. In test mode, background work never runs on its own. It runs only inside /test/clock and /test/drain, so every test is deterministic. Moving the clock never creates evidence: a test that needs 60 days of cycles sends them with their own event times first, then moves the clock.

Endpoint	Behaviour
POST /test/reset	Empty every table
POST /test/clock {now}	Set the server clock. It moves forward only; going backwards returns 409. Before it returns, the server processes every received batch and every retry due up to now, and applies every due transition in event-time order. Each transition is effective at its own due time, never the jump time
POST /test/drain	The same, without moving the clock
POST /test/faults {processing_failures}	The next <i>n</i> processing attempts fail, to exercise retries

Your code must read time only through this clock.

What you build

The business rules above are fixed; how you implement them is up to you. Ten hours is tight, so the scope comes in two tiers.

Required: our tests depend on it	May be a documented compromise
The whole fixed API, including the test endpoints	503 and Retry-After handling
Every rule: gateway, stop and resume, coverage, sensor, late data and corrections	Console filters beyond status, sorting, pagination
Automated tests for the transition rules, including timers, coverage and one late-data correction	UI tests
Fleet page: status and how long it has held ("Disconnected · 2h"), last heartbeat and last qualifying evidence side by side, command state, flags, and the operator actions with reasons	A polished timeline view. A plain list of entries is fine
Sensors page: lifecycle, reason, collection, coverage and next evaluation as separate columns. Unknown, paused and not-checked must look different from zero and from healthy	Situation counts on the dashboard
A console API for those pages, documented in OpenAPI or Markdown	Everything in the console API beyond those pages
A gateway simulator: a CLI that plays at least three failure stories of your choice through the fixed API and the test clock, each ending in an assertion	More than three stories

Put everything you skipped or simplified in `DECISIONS.md`. A documented compromise costs little. A silent one costs a lot.

Stack, change request and hand-in

Item	Expectation
Backend	Python, Django + Django REST Framework, PostgreSQL. No SQLite
Background work	Your choice: Celery, RQ, a DB-backed queue, a management-command loop. Explain it
Frontend	Next.js + TypeScript, any component library
Run	<code>docker compose up</code> starts everything. One command runs your tests, one runs your simulator
Time	About 10 hours. Use any AI tools you like; you must be able to explain and change every line

A change request will arrive four hours after your start, by email, the way real ones do: mid-build. Fit it into the same budget. If you can't finish it, say what is done and what isn't.

If something in this brief is ambiguous, don't stop. Pick an interpretation, write it in `DECISIONS.md` under "Interpretations", and carry on. We only test behaviour this brief defines.

Hand in a git repository with its full, unsquashed history, and:

- `README.md`: how to run the system, the tests and the simulator;
- `DECISIONS.md`, two pages at most: how you model state and time, where each rule lives, how you handle retries, late data and corrections, your interpretations, what you traded off, and what you know is wrong;
- `AI.md`, a few lines: which tools you used, for what, and one thing a tool got wrong that you caught.

We weigh, in this order: whether states and transitions are right under our tests, how you handled the change request, the reasoning in `DECISIONS.md`, the console API design, code and tests, then the UI. After you submit, there's a 75-minute call: you walk us through it, then make one change live.